

Training only the first layer is all you need

Hugo Vidal, intern at the IdePHICS Lab
supervised by Florent Krzakala

14th April - 31st July 2025

Artificial Neural Network :

$$f_{\theta} \longrightarrow \theta = (\theta_1, \dots, \theta_p) \quad p \in \mathbb{N}$$

Artificial Neural Network :

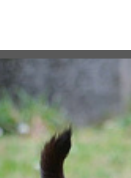
$$f_\theta \longrightarrow \theta = (\theta_1, \dots, \theta_p) \quad p \in \mathbb{N}$$

Training algorithm $\Rightarrow$ Learning a task

Dataset : $\mathcal{D} = \{(\mathbf{x}_\mu, \mathbf{y}_\mu)\}_{\mu=1}^N$

Loss function : $\mathcal{L}(f_{\theta}, \mathcal{D})$

Input Target

A black dog, possibly a Labrador Retriever, is standing in a field of tall grass. The dog is looking upwards towards a large black arrow pointing up, which is positioned above the dog's head. The dog has a yellow collar. The background is a blurred field of green grass and some trees.

↑
Dog

Artificial Neural Network :

$$f_{\theta} \longrightarrow \theta = (\theta_1, \dots, \theta_p) \quad p \in \mathbb{N}$$

Training algorithm $\Rightarrow$ Learning a task

$$\text{Dataset : } \mathcal{D} = \{(\mathbf{x}_{\mu}, \mathbf{y}_{\mu})\}_{\mu=1}^N \quad \Bigg| \quad \text{Loss function : } \mathcal{L}(f_{\theta}, \mathcal{D})$$

$$\Rightarrow \text{ Training step = Minimize the loss : } \quad \theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(f_{\theta}, \mathcal{D}) \quad (\eta \in \mathbb{R})$$

Artificial Neural Network :

$$f_{\theta} \longrightarrow \theta = (\theta_1, \dots, \theta_p) \quad p \in \mathbb{N}$$

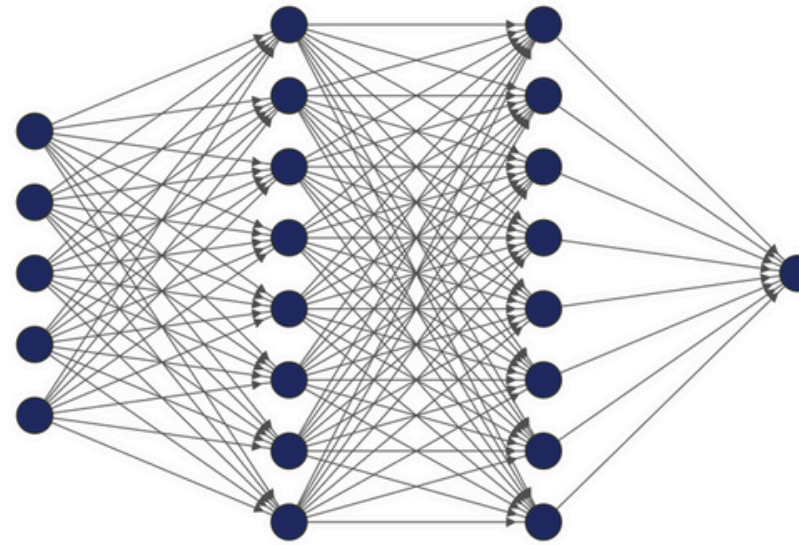
Training algorithm $\Rightarrow$ Learning a task

$$\text{Dataset : } \mathcal{D} = \{(\mathbf{x}_{\mu}, \mathbf{y}_{\mu})\}_{\mu=1}^N \quad \Bigg| \quad \text{Loss function : } \mathcal{L}(f_{\theta}, \mathcal{D})$$

$$\Rightarrow \text{ Training step = Minimize the loss : } \theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(f_{\theta}, \mathcal{D}) \quad (\eta \in \mathbb{R})$$

$$\text{End of training : } \theta^* = \underset{\theta}{\operatorname{argmin}} [\mathcal{L}(f_{\theta}, \mathcal{D})] \quad \Rightarrow \quad \textbf{Generalize well !!}$$

Fully Connected Neural Network (FCNN) :



$$f_{\theta}(\mathbf{x}) = \mathbf{b}_L + W_L \sigma(\mathbf{b}_{L-1} + W_{L-1} \sigma(\dots \sigma(\mathbf{b}_1 + W_1 \mathbf{x})))$$

Non linear function

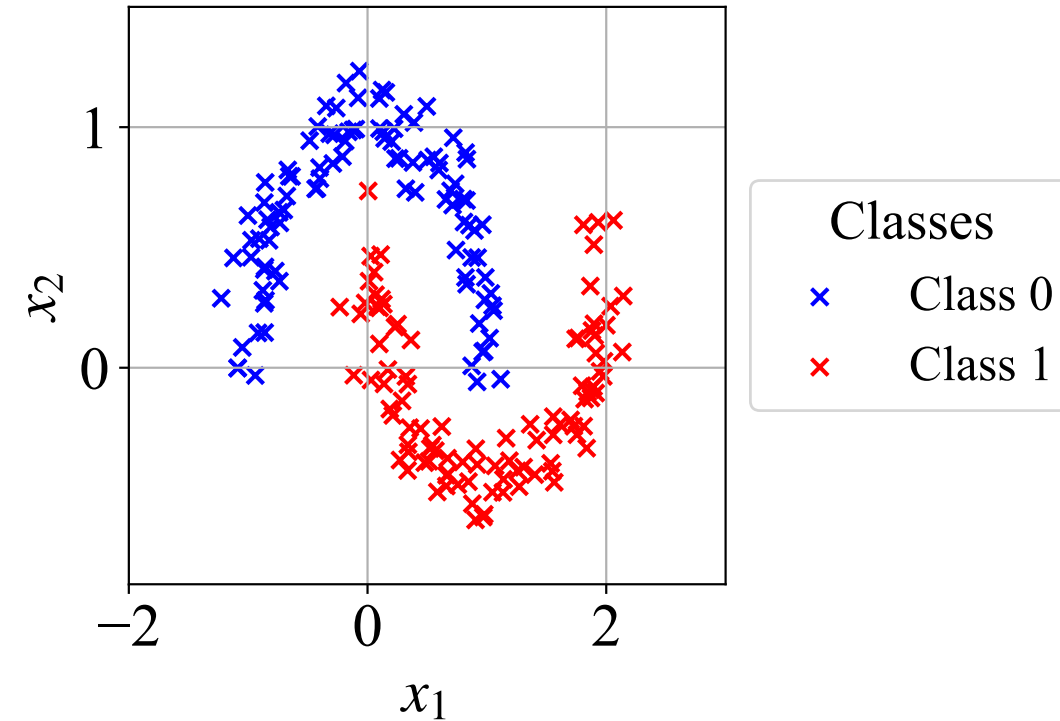
Affine transformation

$$\theta = (W_1, \mathbf{b}_1, \dots, W_L, \mathbf{b}_L)$$

Fully Connected Neural Network (FCNN) :

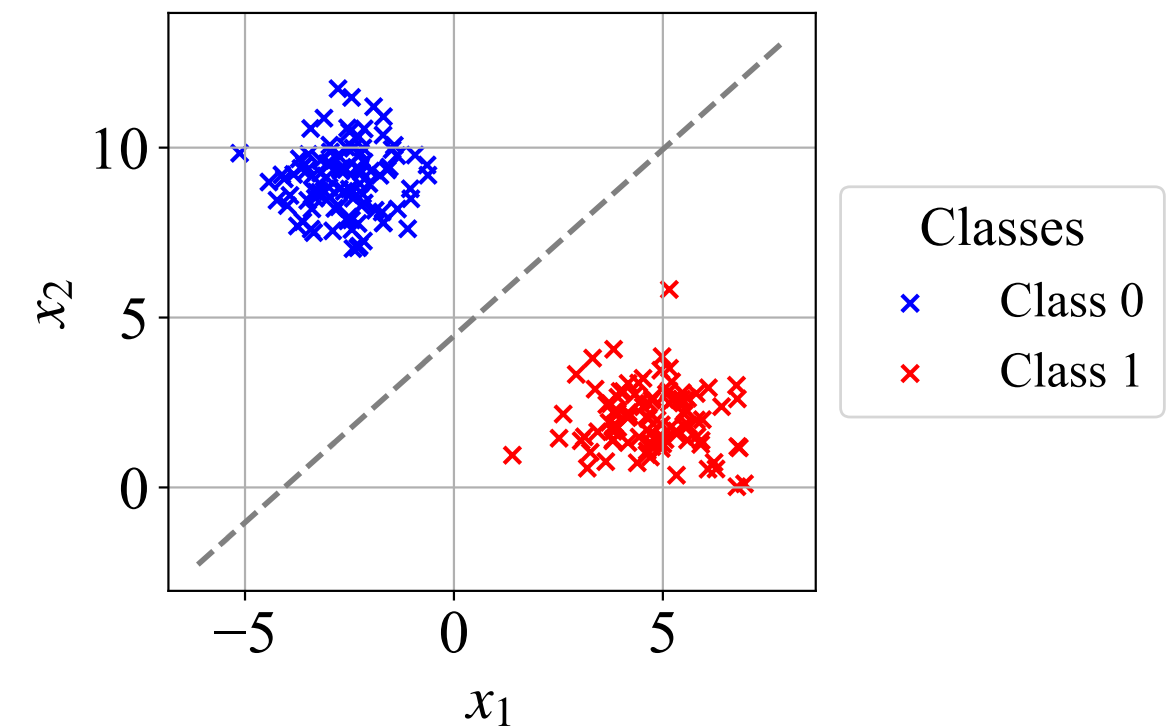
$$f_{\theta}(\mathbf{x}) = \underbrace{\mathbf{b}_L + W_L}_{\text{Linear separation}} \underbrace{\sigma(\mathbf{b}_{L-1} + W_{L-1} \sigma(\dots \sigma(\mathbf{b}_1 + W_1 \mathbf{x})))}_{\text{Feature representation}} \quad \Bigg| \quad \theta = (W_1, \mathbf{b}_1, \dots, W_L, \mathbf{b}_L)$$

Non-linearly separable classes



Learning
→

Linearly separable classes



Is it enough to train only the first layer among the $L-1$ first layers?

Can a FCNN learn meaningful feature representation even if only the first layer is trained ?

- I) Previous works and motivations for the subject
- II) Experimental results on synthetic and real data
- III) Towards a theoretical explanation

The Computational Advantage of Depth: Learning High-Dimensional Hierarchical Functions with Gradient Descent

Yatin Dandi^{1,2}, Luca Pesce¹, Lenka Zdeborová², and Florent Krzakala¹

Hierarchical teacher function :

$$\begin{cases} f^*(\mathbf{x}) = g(\mathbf{a}_{L-1}^\top P_{k,L-1}(\mathbf{h}_{L-2}(\mathbf{x}))) \\ \mathbf{h}_l(\mathbf{x}) = \mathbf{a}_l P_{k,l}(\mathbf{h}_{l-1}(\mathbf{x})) & \forall l \in \llbracket 2, L-2 \rrbracket \\ \mathbf{h}_1(\mathbf{x}) = \mathbf{W}^* \mathbf{x} \end{cases}$$

Synthetic dataset :

Gaussian i.i.d. data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d}$

$$\mathcal{D} = \left\{ \left(\mathbf{x}_\mu, \underset{\substack{\uparrow \\ \text{Teacher}}}{f^*(\mathbf{x}_\mu)}} \right) \right\}_{\mu=1}^N$$

The Computational Advantage of Depth: Learning High-Dimensional Hierarchical Functions with Gradient Descent

Yatin Dandi^{1,2}, Luca Pesce¹, Lenka Zdeborová², and Florent Krzakala¹

Hierarchical teacher function :

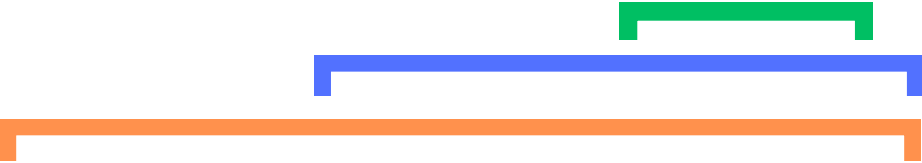
$$\begin{cases} f^*(\mathbf{x}) = g(\mathbf{a}_{L-1}^\top P_{k,L-1}(\mathbf{h}_{L-2}(\mathbf{x}))) \\ \mathbf{h}_l(\mathbf{x}) = \mathbf{a}_l P_{k,l}(\mathbf{h}_{l-1}(\mathbf{x})) & \forall l \in \llbracket 2, L-2 \rrbracket \\ \mathbf{h}_1(\mathbf{x}) = \mathbf{W}^* \mathbf{x} \end{cases}$$

Student FCNN :

$$\hat{f}_\theta(\mathbf{x}) = b_{L'} + \mathbf{w}_{L'}^\top \sigma(\mathbf{b}_{L'-1} + \mathbf{W}_{L-1} \dots \sigma(\mathbf{b}_1 + \mathbf{W}_1 \mathbf{x}))$$

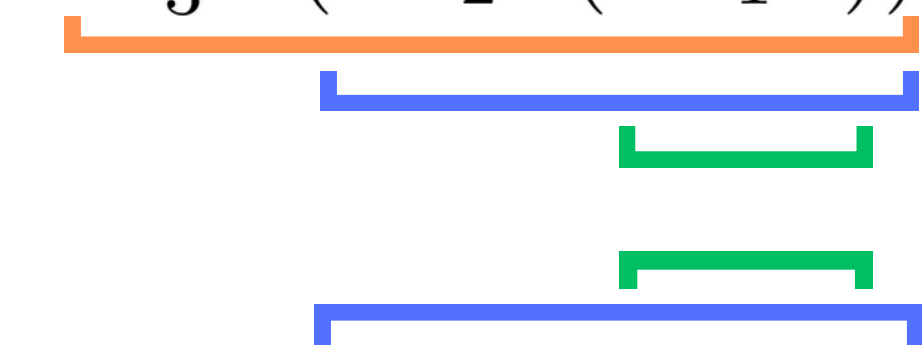
$\Rightarrow L' = L :$ Efficient recovery

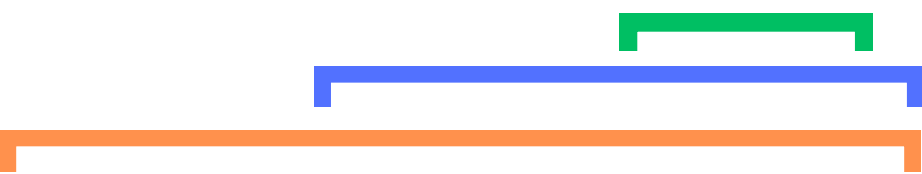
$\Rightarrow L' < L :$ Hard recovery


$$f^*(\mathbf{x}) = \tanh \left(\frac{\mathbf{a}^{\star \top} P_3 (W^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right)$$

Gaussian i.i.d. data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d}$

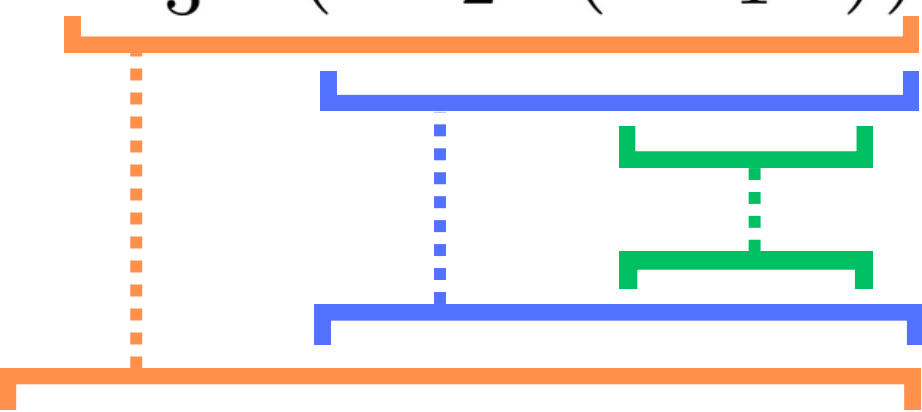
Hierachical learning :

$$\hat{f}_\theta(\mathbf{x}) = \mathbf{w}_3^\top \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$$


$$f^*(\mathbf{x}) = \tanh \left(\frac{\mathbf{a}^{*\top} P_3 (W^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right)$$


Gaussian i.i.d. data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d}$

Hierachical learning :

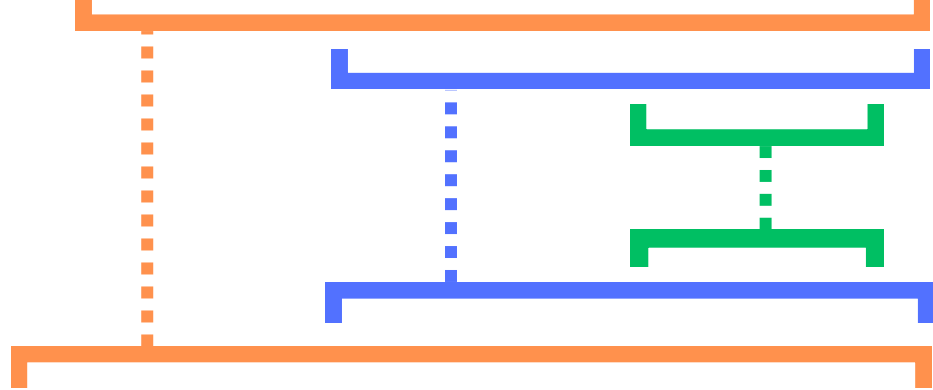
$$\hat{f}_\theta(\mathbf{x}) = \mathbf{w}_3^\top \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$$


$$f^*(\mathbf{x}) = \tanh \left(\frac{\mathbf{a}^{*\top} P_3 (W^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right)$$

Gaussian i.i.d. data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d}$

Hierarchical learning :

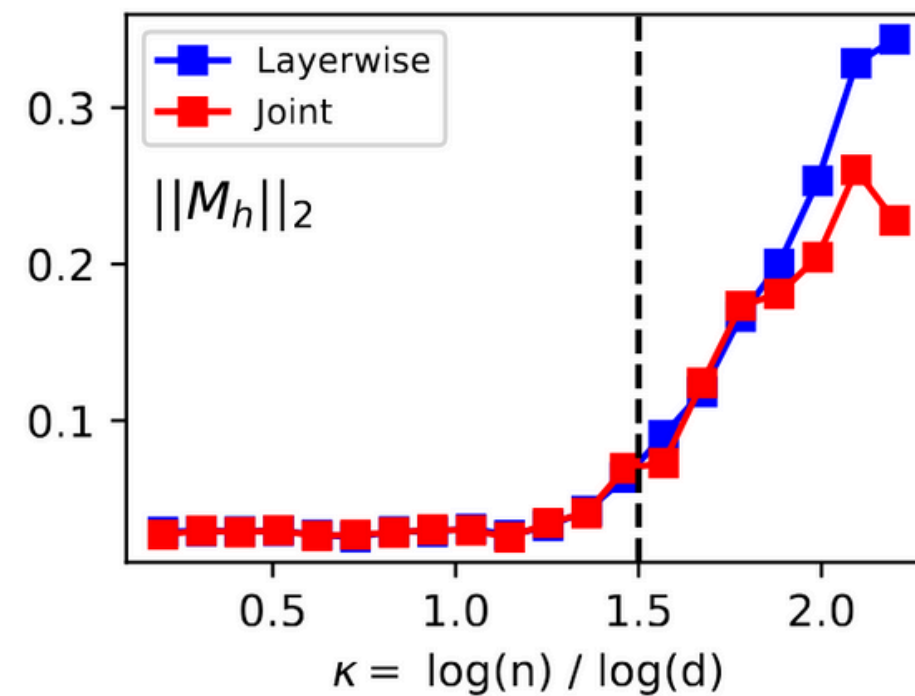
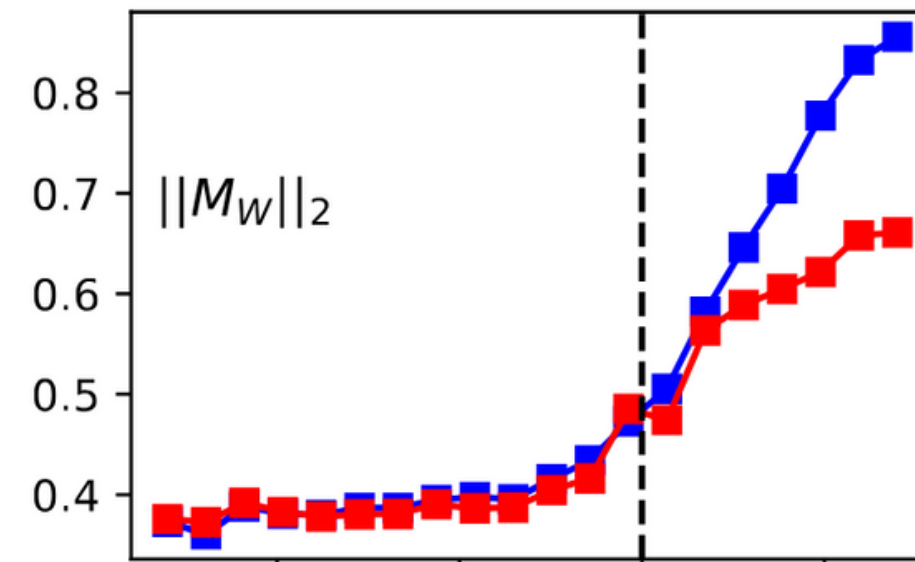
$$\hat{f}_\theta(\mathbf{x}) = \mathbf{w}_3^\top \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$$



$$f^*(\mathbf{x}) = \tanh \left(\frac{\mathbf{a}^{*\top} P_3(\mathbf{W}^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right)$$

Gaussian i.i.d. data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d}$

Numerical results :



$$M_W = \frac{\mathbf{W}_1 \mathbf{W}^{*\top}}{\|\mathbf{W}_1\|_F \|\mathbf{W}^*\|_F}$$

$$M_h = \frac{\mathbb{E}[\mathbf{h}(\mathbf{z}) \mathbf{h}^*(\mathbf{z})]}{\sqrt{\mathbb{E}[\mathbf{h}(\mathbf{z})^2]}}$$

$$\begin{cases} \mathbf{h}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}) \\ \mathbf{h}^*(\mathbf{x}) = \mathbf{a}^\top P_3(\mathbf{W}^* \mathbf{x}) \end{cases}$$

$\Rightarrow$ $\mathbf{W}_1$ is learned to maximize the correlation propagated through the successive layers.

Teacher generated dataset :

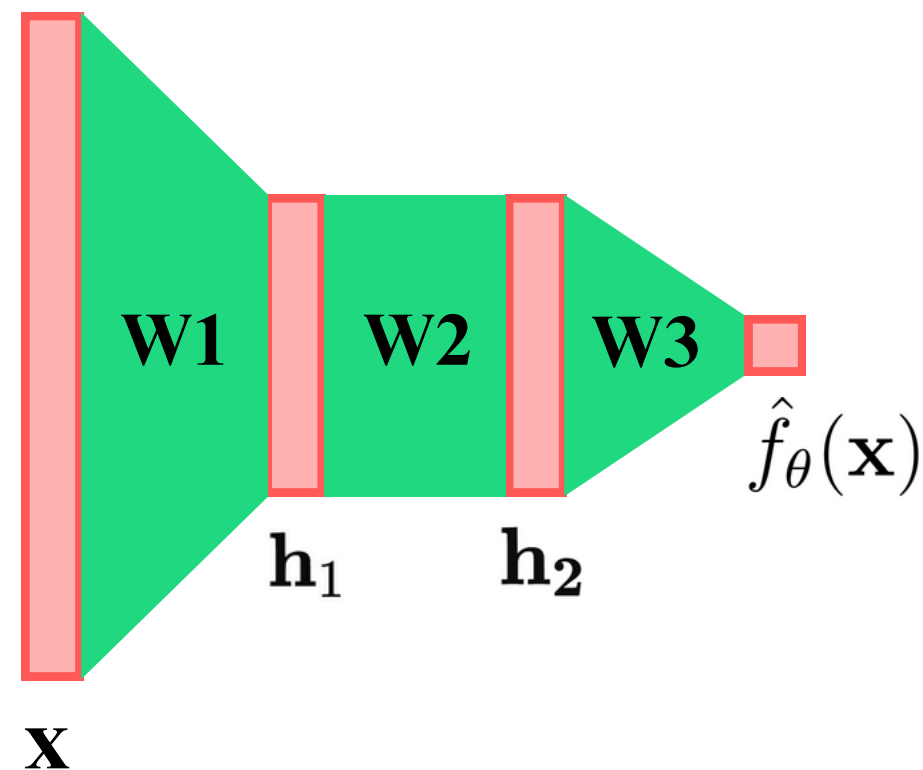
$n = 100k$ datas

$d = 100$

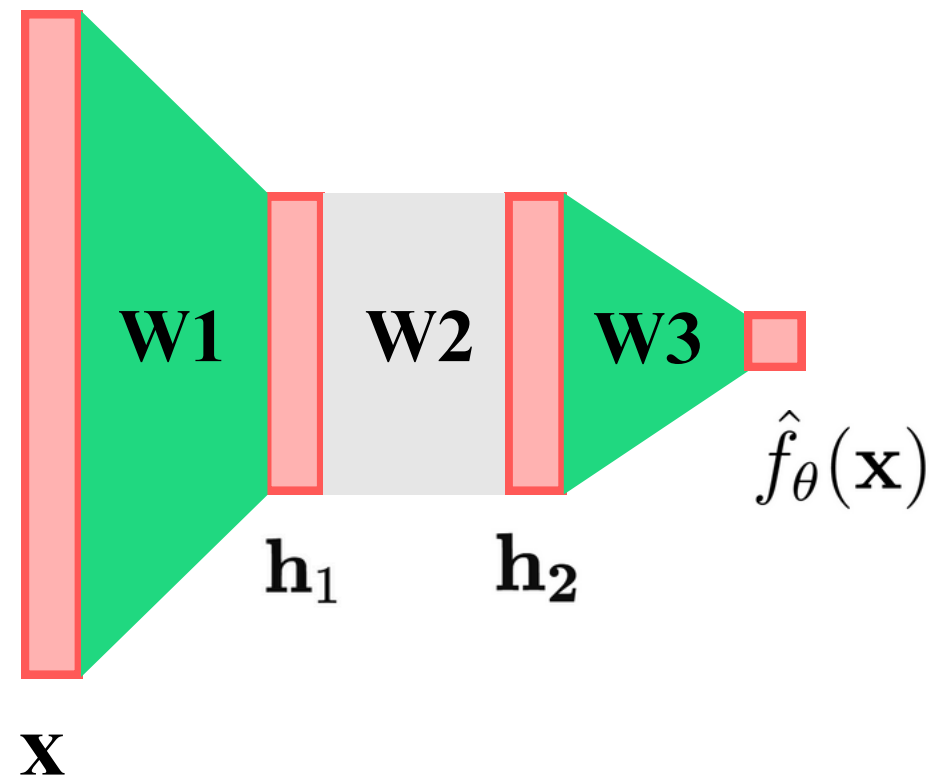
$$\left\{ \begin{array}{l} \text{Gaussian i.i.d. data } \{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d} \\ f^*(\mathbf{x}) = \tanh \left(\frac{3\mathbf{a}^{*\top} P_3(W^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right) \end{array} \right.$$

FCNNs :

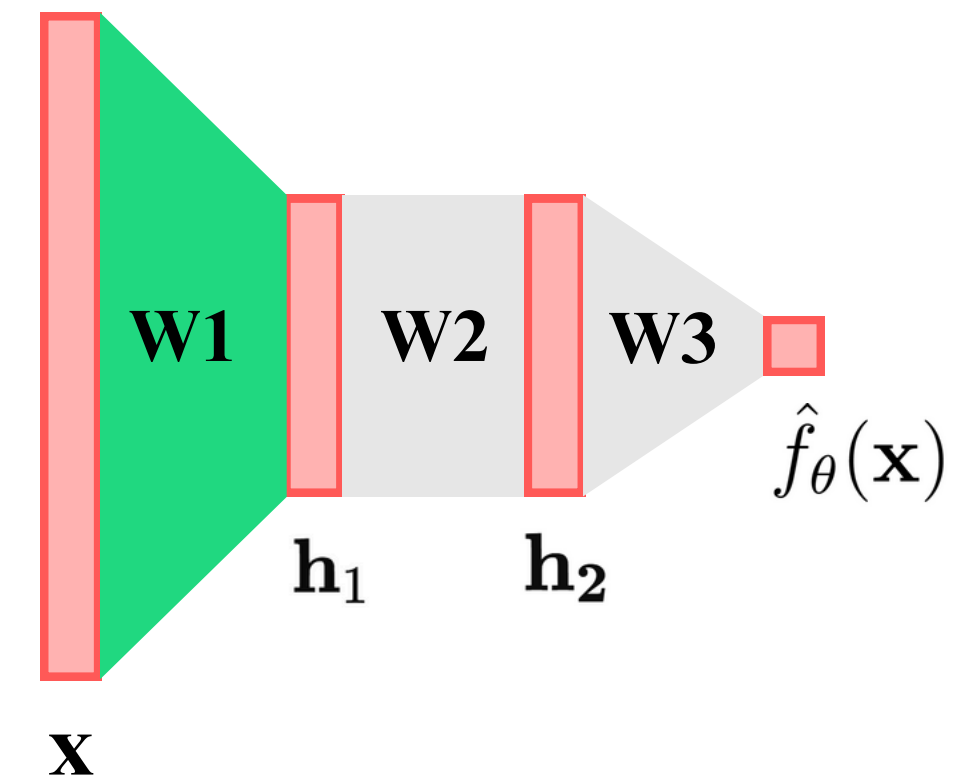
Fully trained



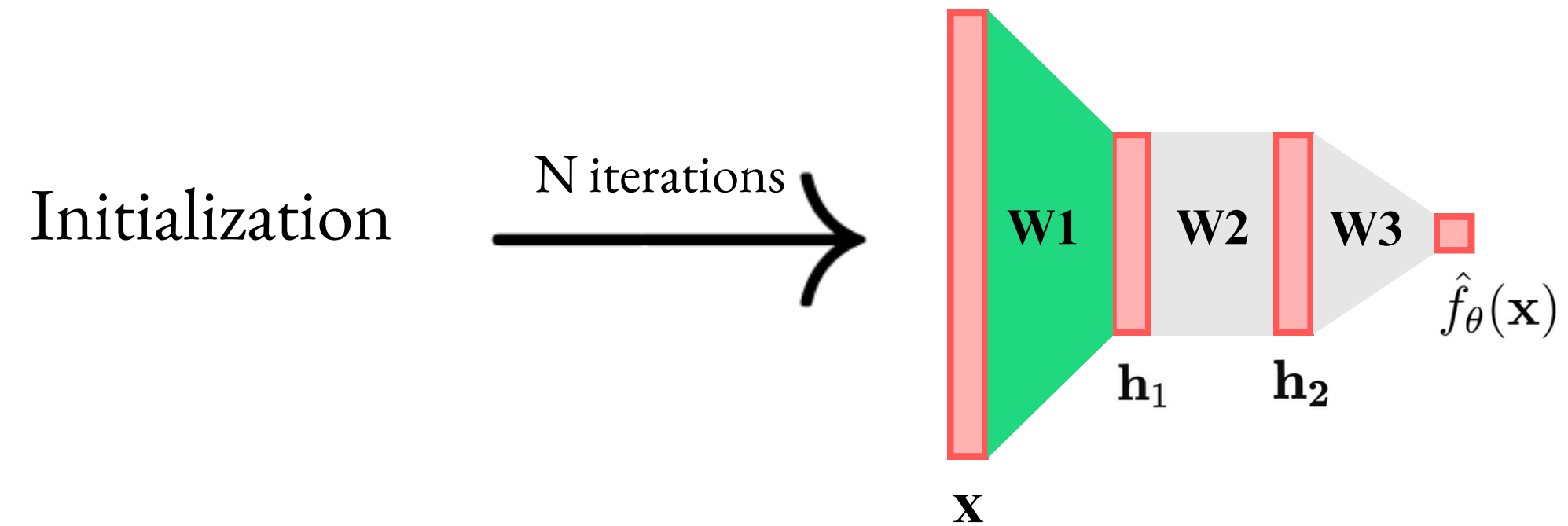
Extremities trained



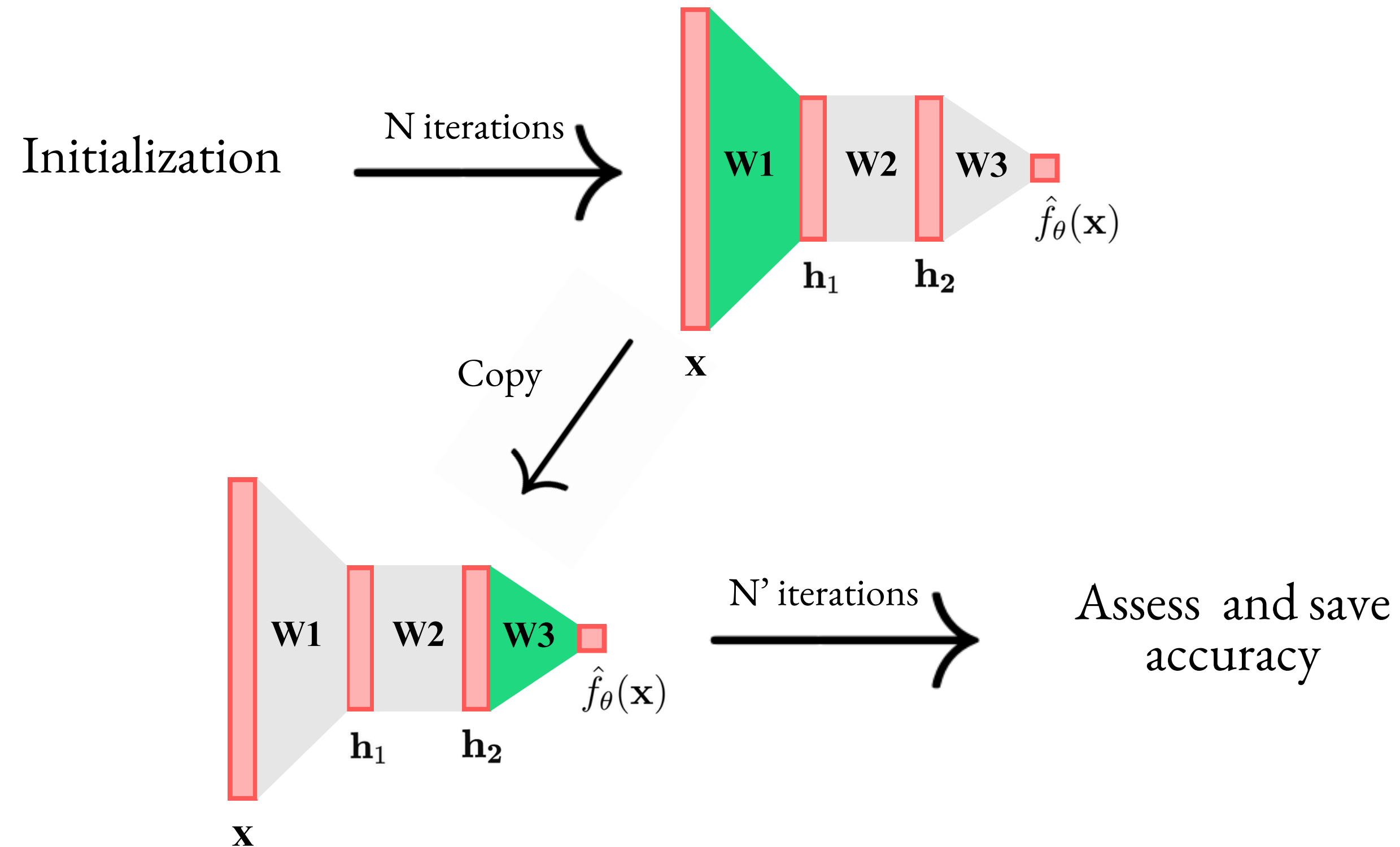
First trained



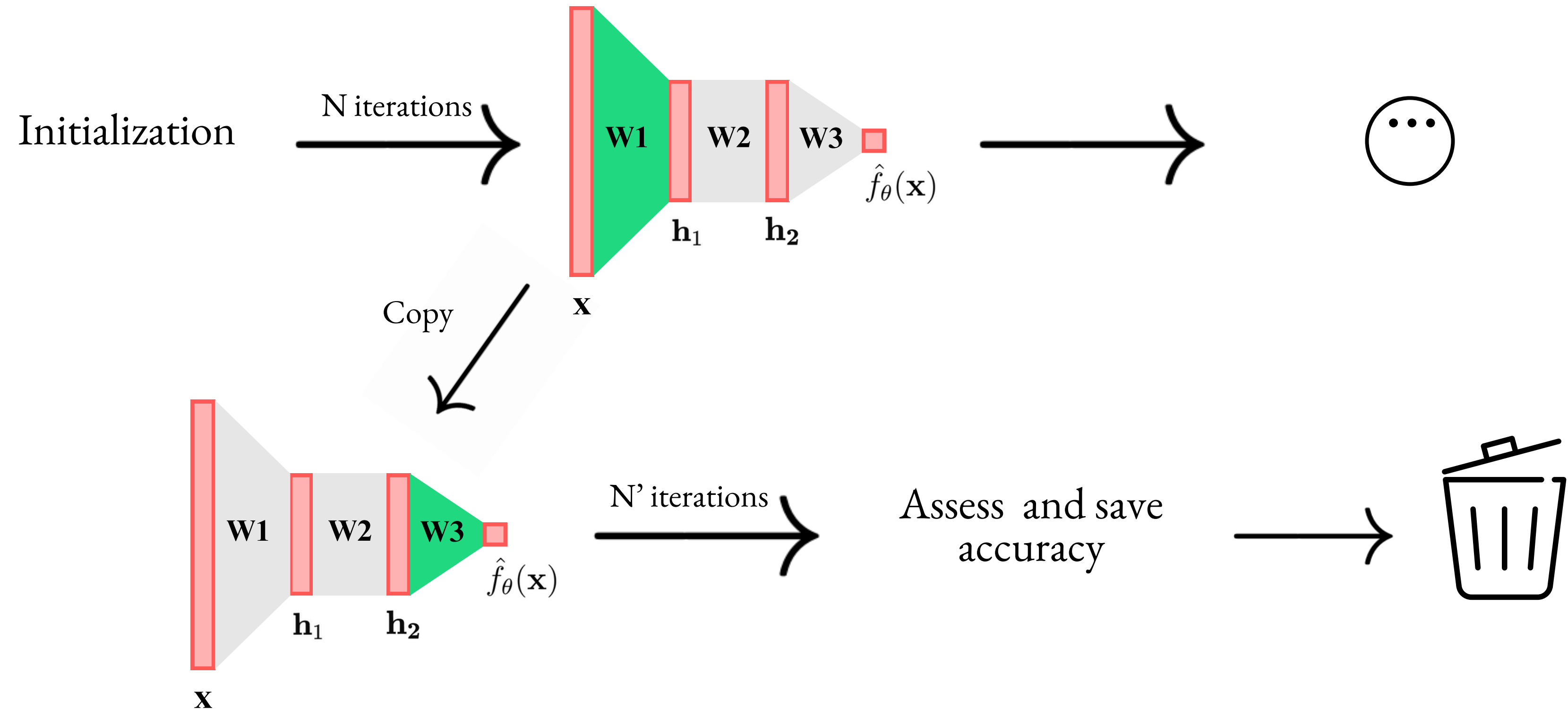
First trained + Last fine tuned :



First trained + Last fine tuned :



First trained + Last fine tuned :



Teacher generated dataset :

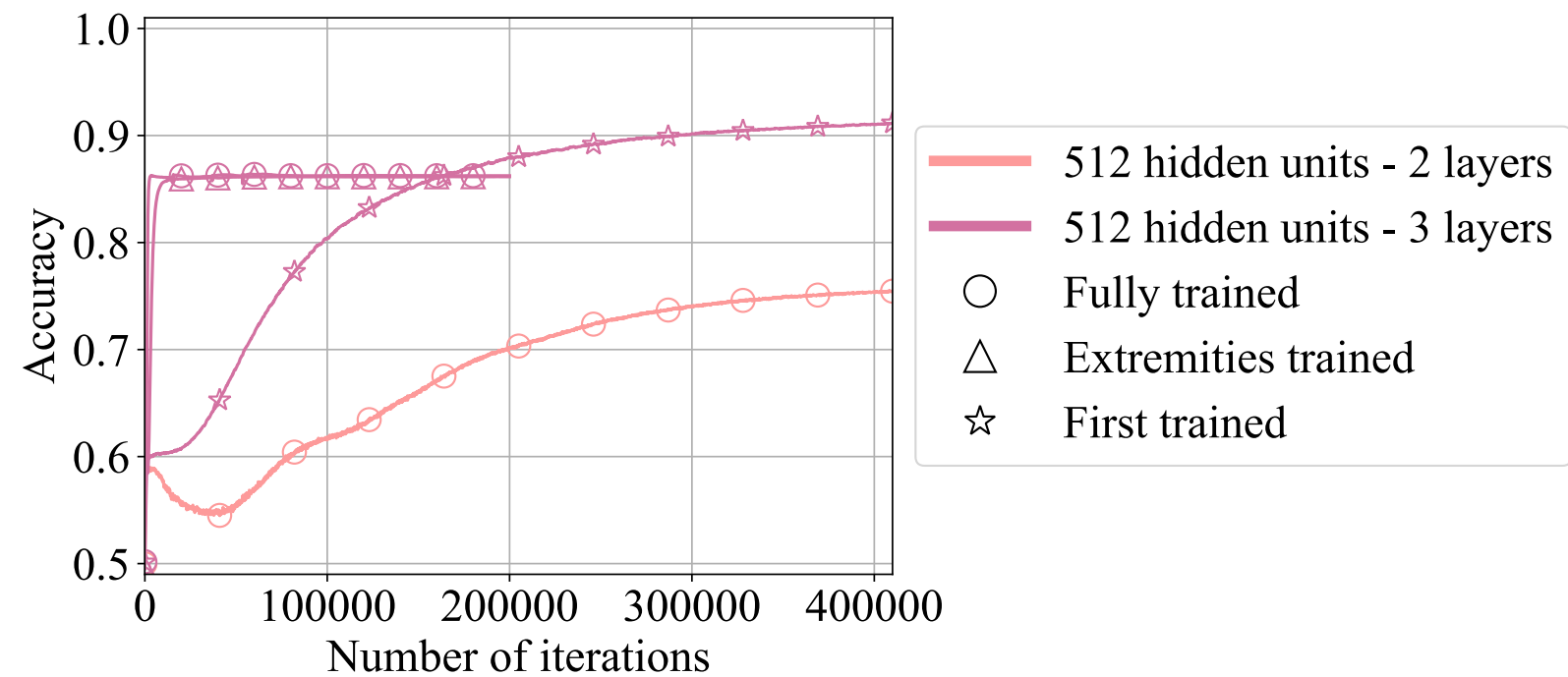
$n = 100k$ datas

$d = 100$

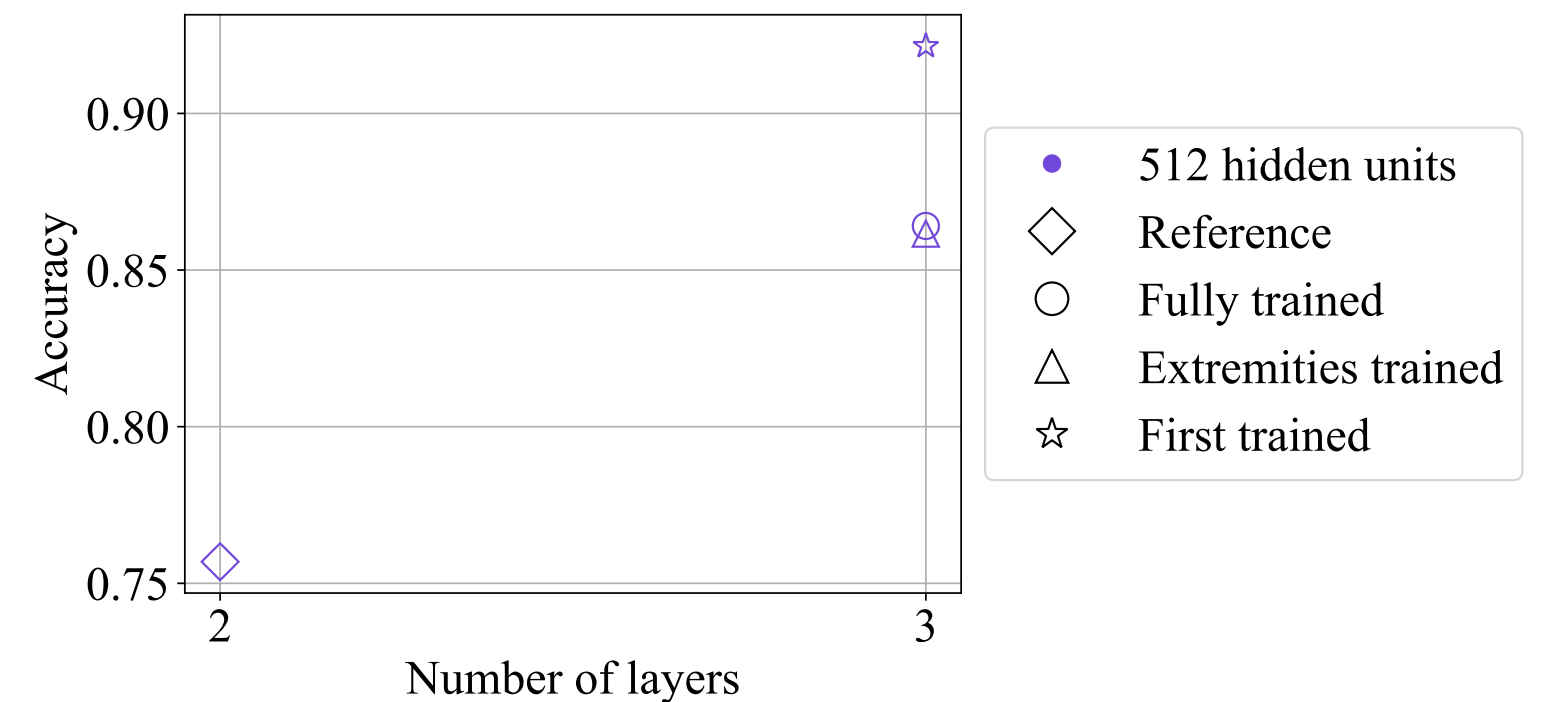
$$\{\mathbf{x}_\mu, f^*(\mathbf{x}_\mu)\}_{\mu=1}^n$$

$$\begin{cases} \text{Gaussian i.i.d. data } \{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^{n \times d} \\ f^*(\mathbf{x}) = \tanh \left(\frac{3\mathbf{a}^{*\top} P_3(W^* \mathbf{x})}{\sqrt{d^{\varepsilon_1=1/2}}} \right) \end{cases}$$

Accuracy during training for the different architectures and types of training



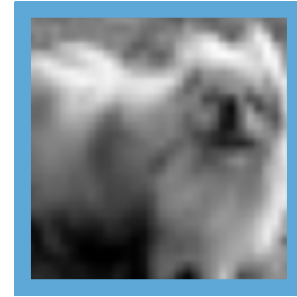
Maximum accuracy for the different architectures and types of training



Dataset :
12k datas
(CIFAR-10)

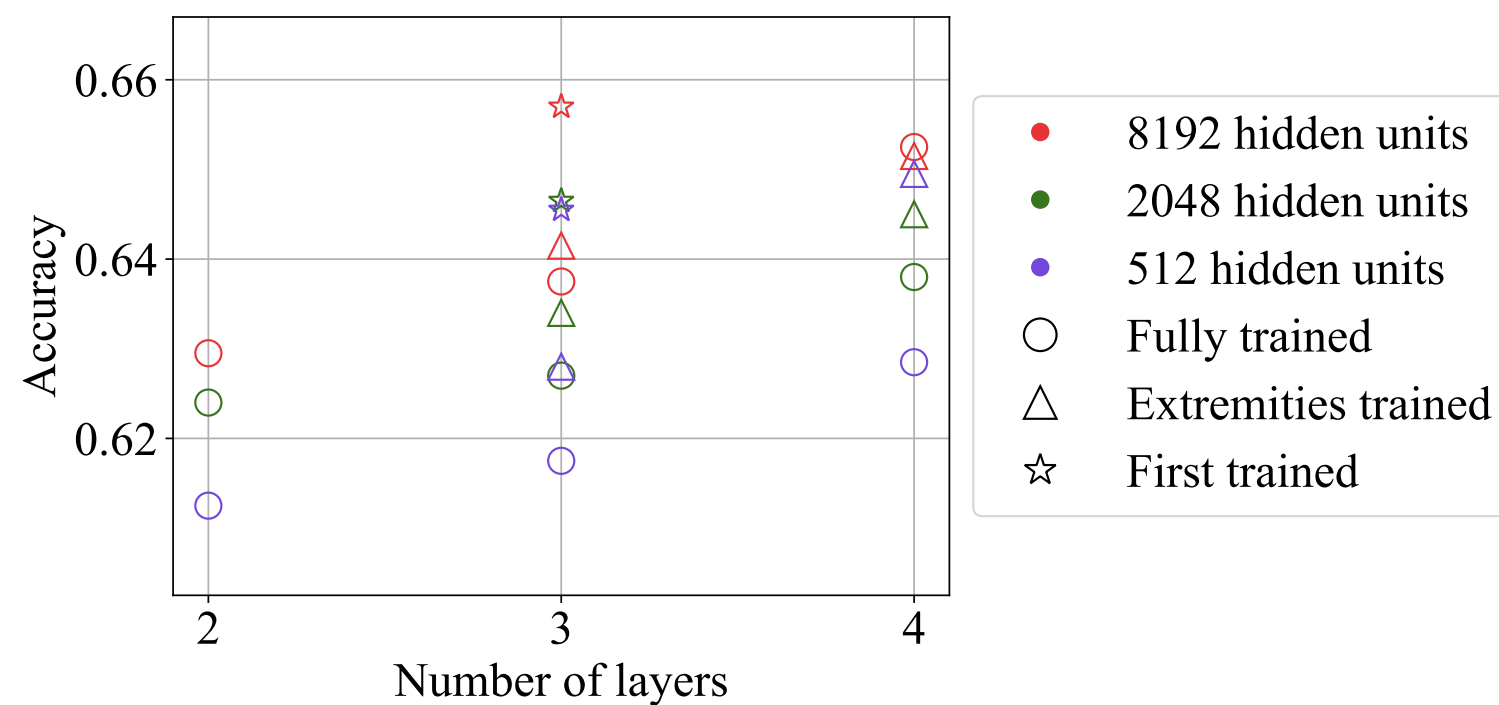


Cat



Dog

Maximum accuracy for
the different architectures and types of training



Dataset :
12k datas
(CIFAR-10)

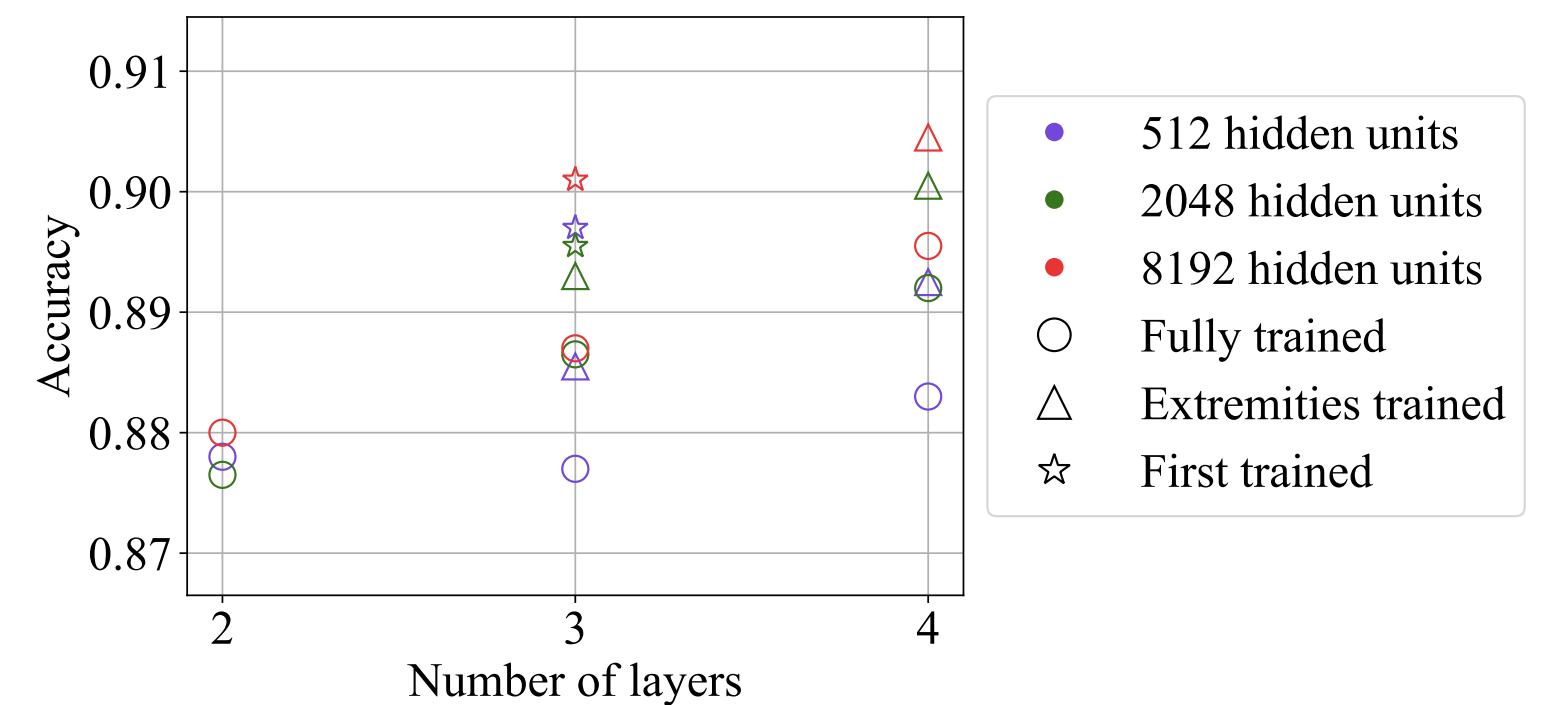


Car



Plane

Maximum accuracy for
the different architectures and types of training



No clear explanation for now

$$f(x) = \mathbf{w}_3^\top \sigma(W_2 \sigma(W_1 x))$$

$$h_{2,i}(x) = \mathbf{w}_{2,i}^\top \sigma(W_1 x)$$

$\Rightarrow$ Let's see how the features of the second layer evolve after a training step on W_1

$$\Delta_{W_1} h_{2,i}(x) = \langle \nabla_{W_1} h_{2,i}(x), \Delta W_1 \rangle \quad (\text{Taylor expansion})$$

$$\Delta W_1 = -\eta \nabla_{\theta} \mathcal{L}(\hat{f}_{\theta}, \mathcal{D}) \approx \frac{\eta}{n} \sum_{\mu=1}^n f^*(x_{\mu}) \cdot \nabla_{W_1} \hat{f}(x_{\mu}) \quad (\text{Correlation loss})$$

$$\Delta h_{2,i}(x) = \frac{\eta}{n} \left(\mathbf{w}_3^\top \left(\sigma'(\mathbf{h}_2(X)) \odot \left[(W_2 \odot W_2^i) \odot (\sigma'(W_1 x) \odot \sigma'(W_1 X)) \right] (W_1 \odot W_1)(x \odot X) \right) \right) \cdot f^*(X)$$

$$X = (x_1, \dots, x_n)$$

⇒ FCNN with only the first and eventually the last layer trained :

- Performs better than a 2 layer FCNN.
- Benefits from depth even if some layers are not trained.

⇒ However there is no clear explanation for the moment.

⇒ This track is promising and still has to be explored.

⇒ If this property generalize to more realistic models :

- It could greatly diminish the duration and (environmental) cost of training.

Thanks !
Any questions ?

$$f^*(\mathbf{x}) = g^* \left(h_{L,1}^*(\mathbf{x}), \dots, h_{L,r}^*(\mathbf{x}) \right),$$

with Gaussian data $\{\mathbf{x}_\mu\}_{\mu=1}^n \in \mathbb{R}^d$, and where each features $\mathbf{h}_\ell^*(\mathbf{x}) \in \mathbb{R}^{d_\ell^\epsilon}$ are recursively defined as:

$$h_{\ell,m}^*(\mathbf{x}) = \frac{1}{\sqrt{d^{\epsilon_{\ell-1}-\epsilon_\ell}}} \mathbf{a}_{\ell,m}^{*\top} P_{k,m,\ell} \left(\mathbf{h}_{\ell-1, \{1+(m-1)d^{\epsilon_{\ell-1}-\epsilon_\ell}, \dots, md^{\epsilon_{\ell-1}-\epsilon_\ell}\}}^*(\mathbf{x}) \right),$$

